

HW1: Mid-term assignment report

Eduardo Luís Pinhal Lopes [103070], v2025-03-26

1	Introduction	1
1.1	Overview of the work	1
1.2	Current limitations	1
2	Product specification	2
2.1	Functional scope and supported interactions	2
2.2	System architecture	2
2.3	API for developers	2
3	Quality assurance	2
3.1	Overall strategy for testing	2
3.2	Unit and integration testing	2
3.3	Functional testing	3
3.4	Code quality analysis	3
3.5	Continuous integration pipeline [optional]	3
4	References & resources	3

1 Introduction

1.1 Overview of the work

This report presents the midterm individual project required for TQS, covering both the software product features and the adopted quality assurance strategy.

The purpose of this work is to develop a full-stack application that allows users to book meals on the Moliceiro University campus. There are several restaurants available and users can select the one they prefer according to their meal preference and the weather forecast. The user can also see the meals planned for each facility until four days in the future and book one and receive a token for that reservation. Tokens can be used by restaurant staff to see if the reservation is valid, cancel the reservation, and be marked as used.

The application is composed of a frontend developed in React and a backend in Spring Boot. To run the application use the docker-compose file. It runs the frontend, the backend, the database, and other two containers that are necessary for the performance tests. All API endpoints are documented using OpenAPI 3.0.

1.2 Current limitations

All basic features were implemented. Some features could have improved validation logic, guaranteeing that there is no way to break the application flow. Also, feature G is implemented. Meals have a reservation limit and group reservations are permitted but only for the same meal. However, the reservation limit is static and doesn't show the real number of reservations allowed. The cache implemented is a bit simple and should be changed to a more appropriate tool for a bigger production scenario. The application could have more tests. The repositories were tested but they do not appear in the final version (in my GitHub repository) due to problems when I integrated the MySQL instance (I didn't find a solution for this problem).

2 Product specification

2.1 Functional scope and supported interactions

The application was developed with two types of users in mind: the clients of the restaurant facilities available on the campus, and the restaurant staff. The user can see all the restaurants available on campus, select one they pretend and get the meals planned for the next four days. They can also see the weather forecast for each day, to aid in their decision, and reserve a meal for several people if the number of people is inferior or equal to the number of meals available.

The staff use the application to verify if the reservation tokens are valid and also to mark them as used or even cancel them.

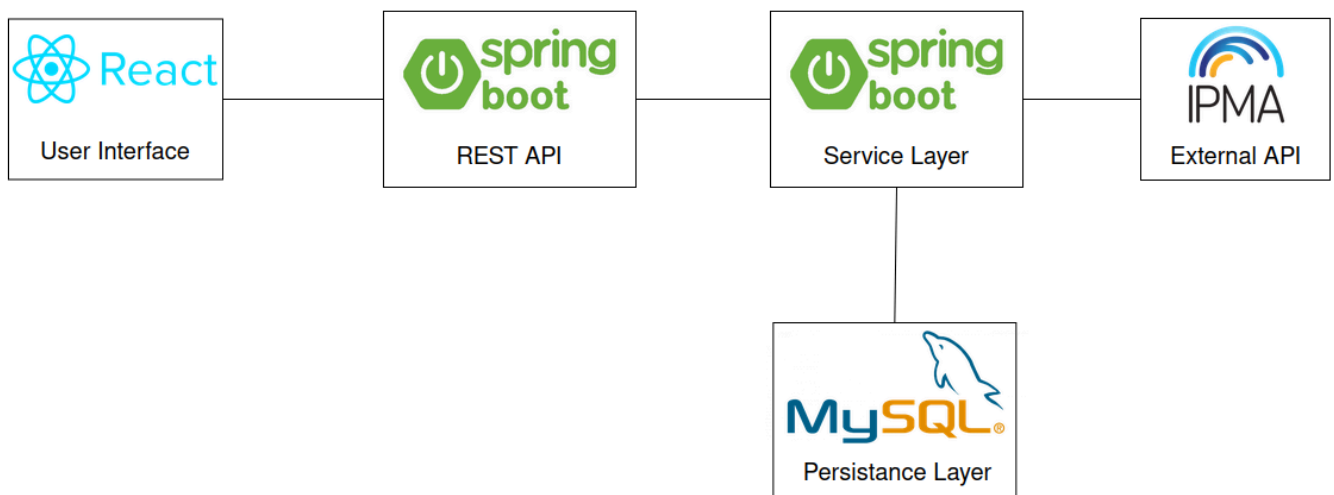
User steps:

1. Open the application and select one of the restaurants available (click on the "View Menus" button) on the home page;
2. See all the meals planned and the weather forecast for the following four days.
3. Select a meal and insert the number of people. Next, click on the make reservation button.
4. Save the token shown in the alert pop-up and click on the "ok" button.

Staff steps:

1. Open the app and select the tab “Reservations”.
2. Insert the given token and click on the button to check if it's valid.
3. Click on the button “used” to mark the token as used or click on the button “cancel” to cancel the reservation.

2.2 System implementation architecture



```

Restaurant v {
  id          integer($int64)
  name        string
  location    string
  cityId      integer($int32)
}
  
```

```

Meal v {
  id          integer($int64)
  name        string
  soup        string
  dessert     string
  mainCourse  string
  mealType    string
  date        string($date)
  reservationLimit integer($int32)
  restaurant  Restaurant > {...}
}
  
```

```

Reservation v {
  id          integer($int64)
  token       string
  reservationDateTime string($date-time)
  numberOfPeople integer($int32)
  meal        Meal > {...}
  valid       boolean
  cancelled   boolean
  used        boolean
}
  
```

```

CacheEntry v {
  minTemperature    number($double)
  maxTemperature    number($double)
  precipitationProbability number($double)
  timestamp         string($date-time)
  forecastDate      string($date)
}
  
```

2.3 API for developers

Servers	
http://localhost:8080 - Generated server url	
restaurant-controller	
GET	/api/v1/restaurants
POST	/api/v1/restaurants
GET	/api/v1/restaurants/{id}
reservation-controller	
POST	/api/v1/reservations/{mealId}
POST	/api/v1/reservations/used/{token}
POST	/api/v1/reservations/cancel/{token}
GET	/api/v1/reservations/{token}
meal-controller	
GET	/api/v1/meals
POST	/api/v1/meals
GET	/api/v1/meals/{id}
GET	/api/v1/meals/restaurant/{restaurantId}
weather-controller	
GET	/api/v1/weather/{cityId}
GET	/api/v1/weather/statistics
GET	/api/v1/weather/all/{cityId}

3 Quality assurance

3.1 Overall strategy for testing

I started the development by creating the four entities (that are above) because they are the base of the project. I tried following the Test-Driven Development (TDD) development strategy, known for being a good way of ensuring software quality. However, some methods were implemented before writing the tests. A good example of this is the weather service and the weather cache (in these classes I preferred trying first to implement the methods and then evaluate my solution with tests). In other classes, I tried to write the test first, but these were never the final version. In most cases, I added to refactor the test because, with the implementation, my ideas for some methods changed. All methods used were presented and lectured in practical and theoretical classes. The tools used to test the backend were JUnit5 for unit tests, Mockito, and MockMvc for unit and integration tests (services and controllers). For the frontend, Cucumber and Selenium were used in a Behavior-Driven Development (BDD) approach.

3.2 Unit and integration testing

Unit tests were used to ensure the behavior of the entities, cache, and repositories (as discussed before, it doesn't appear in the final version available on GitHub) was correct and exactly as expected.

Service level tests were implemented with dependency isolation to guarantee that services were tested without having to worry about the repositories or the external API (IPMA).

Integration tests were implemented to test the behavior of the controllers. For each endpoint, there is a test to validate its behavior.

```
public class WeatherCacheTest {
    private WeatherCache cache;
    private LocalDate testDate;
    private CacheEntry validEntry;

    @BeforeEach
    void setUp() {
        cache = new WeatherCache(ttlMinutes:10); // 10 minute TTL
        testDate = LocalDate.now();
        validEntry = new CacheEntry(minTemperature:12.5, maxTemperature:25.0, precipitation:0.1, LocalDateTime.now(), testDate);
    }

    @Test
    void testPutAndGet() {
        cache.put(testDate, validEntry);
        CacheEntry result = cache.get(testDate);
        assertNotNull(result);
        assertEquals(validEntry.getMinTemperature(), result.getMinTemperature());
        assertEquals(expected:1, cache.getHits());
        assertEquals(expected:0, cache.getMisses());
        assertEquals(expected:1, cache.getPuts());
    }

    @Test
    void testGetReturnsNullIfExpired() {
        CacheEntry expiredEntry = new CacheEntry(minTemperature:10, maxTemperature:20, precipitation:0.5,
            LocalDateTime.now().minusMinutes(15), testDate); // expired

        cache.put(testDate, expiredEntry);
        CacheEntry result = cache.get(testDate);
        assertNull(result);
        assertEquals(expected:0, cache.getHits());
        assertEquals(expected:1, cache.getMisses());
    }

    @Test
    void testContainsKey() {
        cache.put(testDate, validEntry);
        assertTrue(cache.containsKey(testDate));
    }

    @Test
    void testClearAll() {
        cache.put(testDate, validEntry);
        cache.clearAll();
        assertFalse(cache.containsKey(testDate));
    }
}
```

```

@ExtendWith(MockitoExtension.class)
@MockitoSettings(strictness = Strictness.LENIENT)
class ReservationService_UnitTest {

    @Mock
    private ReservationRepo reservationRepo;

    @Mock
    private MealService mealService;

    @InjectMocks
    private ReservationService reservationService;

    private Meal meal;
    private Reservation reservation1;
    private Reservation reservation2;

    @BeforeEach
    void setUp() {
        meal = new Meal(name:"Lunch A", soup:"Tomato soup", dessert:"Apple pie", mainCourse:"Grilled chicken",
            mealType:"lunch", LocalDate.now(), restaurant:null, reservationLimit:10);
        meal.setId(1L);

        reservation1 = new Reservation(numberOfPeople:3, meal);
        reservation2 = new Reservation(numberOfPeople:2, meal);

        when(mealService.getMealById(id:1L)).thenReturn(meal);
        when(mealService.getMealById(-1L)).thenReturn(value:null);
        when(reservationRepo.findByMealId(mealId:1L)).thenReturn(Arrays.asList(reservation1, reservation2));
    }

    @Test
    void whenValidReservation_thenItShouldBeSaved() {
        Reservation newRes = new Reservation(numberOfPeople:4, meal);
        when(reservationRepo.save(any(type:Reservation.class))).thenReturn(newRes);

        Reservation saved = reservationService.makeReservation(mealId:1L, numberOfPeople:4);
        assertThat(saved).isNotNull();
        assertThat(saved.getNumberOfPeople()).isEqualTo(expected:4);

        verify(mealService, times(wantedNumberOfInvocations:1)).getMealById(id:1L);
        verify(reservationRepo, times(wantedNumberOfInvocations:1)).findByMealId(mealId:1L);
        verify(reservationRepo, times(wantedNumberOfInvocations:1)).save(any(type:Reservation.class));
    }
}

```

```

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class WeatherControllerIT {

    @LocalServerPort
    private int randomServerPort;

    @Autowired
    private TestRestTemplate restTemplate;

    private final int cityId = 1010500;
    private final LocalDate targetDate = LocalDate.now().plusDays(1);

    @Test
    void whenValidDateAndCity_thenReturnsForecast() throws Exception {
        String url = "http://localhost:" + randomServerPort + "/api/v1/weather/" + cityId + "?date=" + targetDate;

        ResponseEntity<CacheEntry> response = restTemplate.exchange(
            url, HttpMethod.GET, null, CacheEntry.class);

        assertThat(response.getStatusCode()).isEqualTo(HttpStatus.OK);
        assertThat(response.getBody()).isNotNull();
        assertThat(response.getBody().getMinTemperature()).isGreaterThan(other:0);
        assertThat(response.getBody().getMaxTemperature()).isGreaterThan(other:0);
        assertThat(response.getBody().getPrecipitationProbability()).isBetween(start:0.0, end:100.0);
    }

    @Test
    void whenInvalidDate_thenReturnsEmpty() throws Exception {
        LocalDate invalidDate = LocalDate.of(2025, 12, 25);

        String url = "http://localhost:" + randomServerPort + "/api/v1/weather/" + cityId + "?date=" + invalidDate;

        ResponseEntity<String> response = restTemplate.exchange(
            url, HttpMethod.GET, null, String.class);

        assertThat(response.getStatusCode()).isEqualTo(HttpStatus.OK);
        assertThat(response.getBody()).isNull();
    }
}

```

3.3 Functional testing

For the functional tests, Behavior-Driven Development (BDD) was achieved by using the Selenium WebDriver and Cucumber to represent the workflow of the user. However, only one test scenario was implemented.

Feature: reserve a meal

Scenario: reserve a meal for lunch

Given I am on the home page

When I select the restaurant, clicking on the view menus button

And I click on the reservation button

Then I should confirm my reservation

```

@ExtendWith(SeleniumJupiter.class)
public class ReservationSteps {

    private WebDriver driver = new ChromeDriver();

    @Given("I am on the home page")
    public void iAmOnTheHomePage() {
        driver.get(url:"http://localhost:3030");
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
    }

    @When("I select the restaurant, clicking on the view menus button")
    public void iSelectTheRestaurantClickingOnTheViewMenusButton() {
        WebElement viewMenusButton = driver.findElement(By.linkText(linkText:"View Menus"));
        viewMenusButton.click();
    }

    @When("I click on the reservation button")
    public void iClickOnTheReservationButton() {
        WebElement reserveButton = driver.findElement(By.cssSelector([
            cssSelector:".mb-5:nth-child(2) .col-md-6:nth-child(1) .btn"
        ]));
        reserveButton.click();
    }

    @Then("I should confirm my reservation")
    public void iShouldConfirmMyReservation() {
        WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
        wait.until(ExpectedConditions.alertIsPresent()); // Wait until the alert is present

        String alertText = driver.switchTo().alert().getText();
        assertTrue(alertText.contains("Reservation successful!"));
        driver.switchTo().alert().accept();
    }
}

```

3.4 Non functional testing

For the non functional tests, K6 Grafana and Lighthouse were used. The first one was used to do load tests on the reservation endpoint. The second, was used to measure the performance, quality, and correctness of our app.


```

TOTAL RESULTS

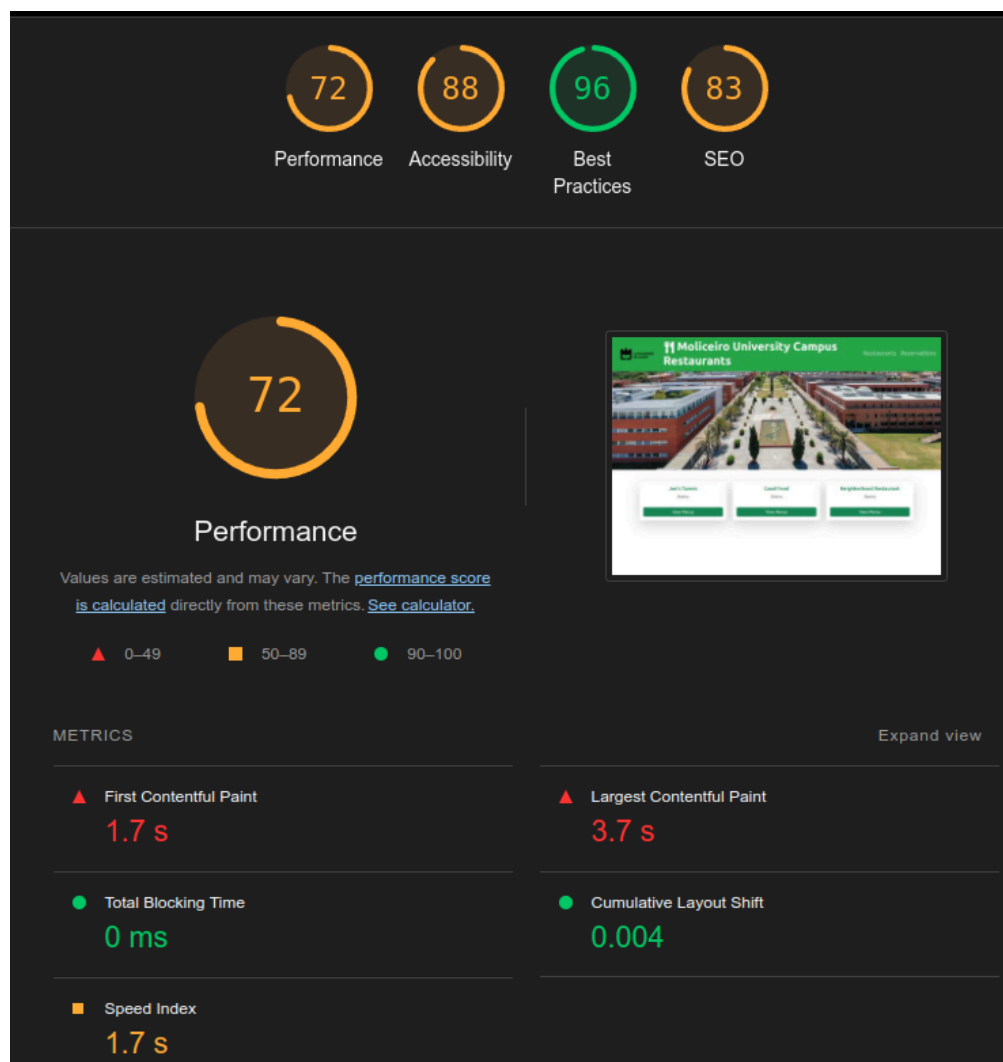
HTTP
  http_req_duration.....: avg=4.28ms min=925.51µs med=4.01
ms max=23.15ms p(90)=6.74ms p(95)=8.09ms
    { expected_response:true }.....: avg=4.28ms min=925.51µs med=4.01
ms max=23.15ms p(90)=6.74ms p(95)=8.09ms
  http_req_failed.....: 0.00% 0 out of 68542
  http_reqs.....: 68542 3427.04892/s

EXECUTION
  iteration_duration.....: avg=4.39ms min=985.54µs med=4.11
ms max=24.06ms p(90)=6.87ms p(95)=8.22ms
  iterations.....: 68542 3427.04892/s
  vus.....: 1 min=1 max=20
  vus_max.....: 20 min=20 max=20

NETWORK
  data_received.....: 11 MB 556 kB/s
  data_sent.....: 11 MB 538 kB/s

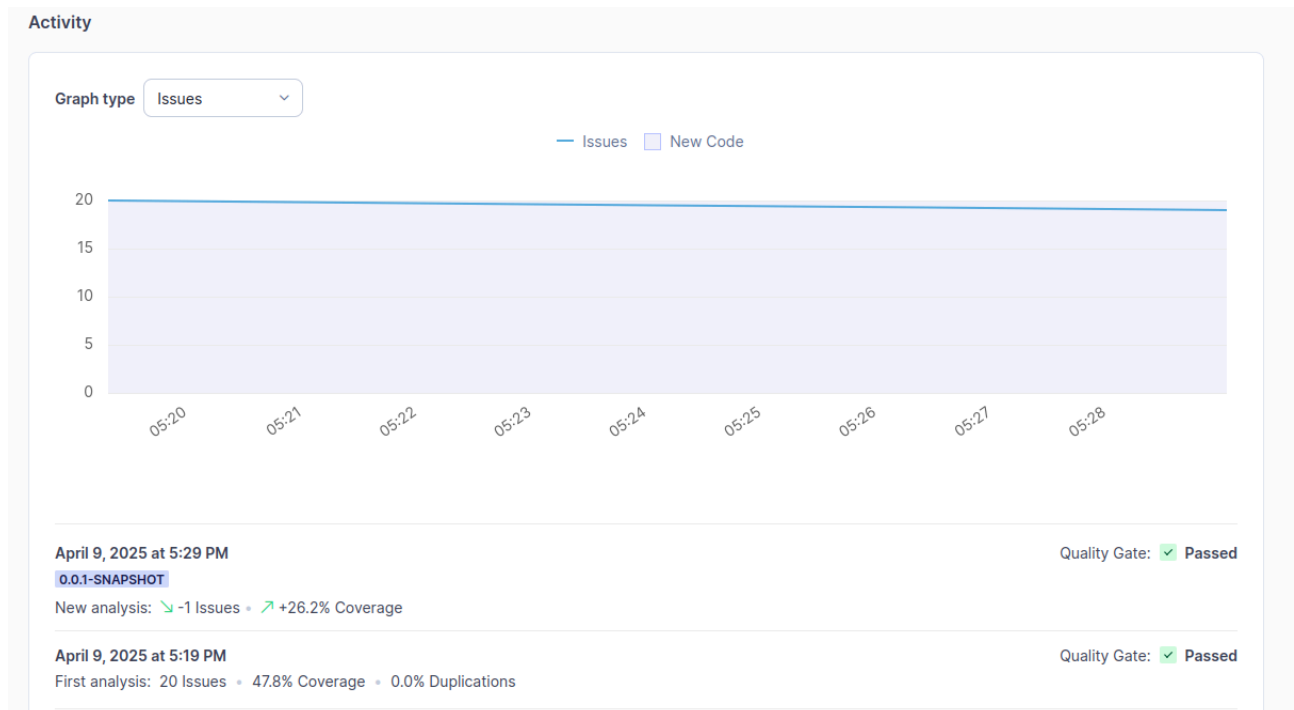
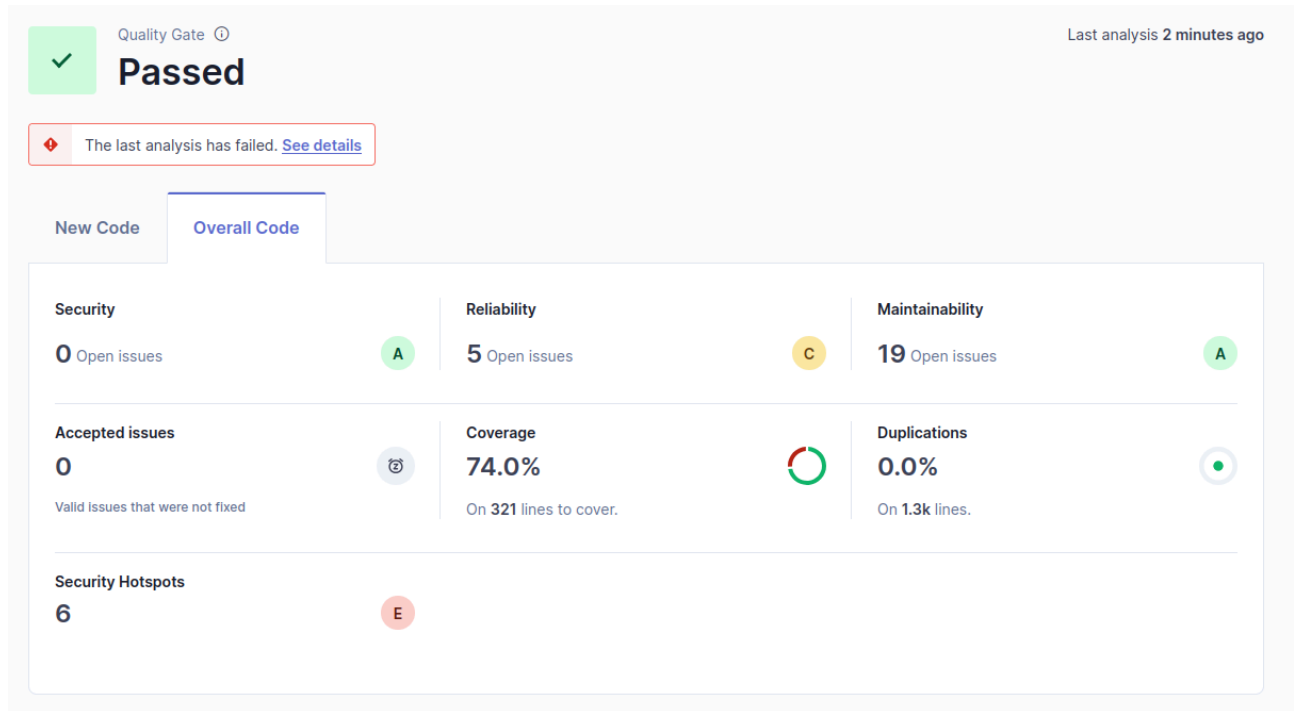
running (20.0s), 00/20 VUs, 68542 complete and 0 interrupted iterations
default ✓ [=====] 00/20 VUs 20s

```



3.5 Code quality analysis

For the code quality analysis, SonarQube was used (with Jacoco) in order to inspect the code for Code Smells, Security Hotspots and Percentage of Code Coverage. Initially, the coverage was low but after some refactoring the percentage increases significantly (this value could be even higher with the repository tests).



3.6 Continuous integration pipeline [optional]

There was no established continuous integration pipeline.

4 References & resources

Project resources

Resource:	URL/location:
Git repository	https://github.com/odraude23/TQS_103070
Video demo	https://github.com/odraude23/TQS_103070/blob/main/HW1-103070/media/demo.webm
QA dashboard (online)	Not Done
CI/CD pipeline	Not Done
Deployment ready to use	Not Done

Reference materials

- IPMA API: <https://api.ipma.pt/>
- F. Kuzman, "Using Swagger 3 in Spring Boot 3 - Fady Kuzman - Medium," *Medium*, Jan. 29, 2023. <https://medium.com/@f.s.a.kuzman/using-swagger-3-in-spring-boot-3-c11a483ea6dc>